

5

QUEUES

OBJECTIVES

After completing this chapter, the reader will be able to understand the following:

- Restricted linear lists—queues
- Implementation of queues using arrays
- Implementation of circular queues
- Use of queues in simulations, job scheduling, and other applications

We have studied linear data structures, namely, arrays and stacks. In arrays, element insertion at and deletion from any position causes a lot of data movement. On the other hand, in stacks, these operations are performed at only one end, the *top*. A *queue* is a special type of data structure that performs insertions at one end called the *rear* and deletions at the other end called the *front*. Let us discuss the concept and functioning of queues in this chapter.

5.1 CONCEPT OF QUEUES

In our daily life, we have experienced standing in queues for various reasons such as purchasing tickets or getting admission to educational institutes. In all such places, we have to wait in a queue for our turn to get the service.

Similarly a queue is a common example of a linear list or an ordered list where data can be inserted at and deleted from different ends. The end at which data is inserted is called the *rear* and that from which it is deleted is called the *front*. These limits guarantee that the data is processed in the sequence in which they are entered. In short, a queue is a *first in first out* (FIFO) or *last in last out* (LILO) structure.

Consider an ordered list $L = \{a_1, a_2, a_3, a_4, \dots, a_n\}$. If we assume that L represents a queue, then a_1 is the front-end element and a_n is the rear-end element. In addition, a_i is behind a_{i-1} .

Let us consider a queue Q of customers standing at a ticket counter.

$Q = \{\text{Shweta, Anup, Saurabh, Vishnu, Shivadmika, Alan, Devanarayanan, Anagha}\}$

In the queue Q , Shweta is at the front end and Anagha is at the rear end.

Queues are one of the most common data processing structures. They are frequently used in most system software such as operating systems, network and database implementations, and other areas. Queues are very useful in time-sharing and distributed computer systems where many widely distributed users share the system simultaneously. Whenever a user places a request, the operating system adds the request at the end of the queue of jobs waiting to be executed. The CPU executes the job at the front of the queue.

5.2 QUEUE AS ABSTRACT DATA TYPE

Look at the queue at the bus stop in Fig. 5.1. Here, the person to get inside the bus is the one who is at the front. The new person joining would stand at the rear end.

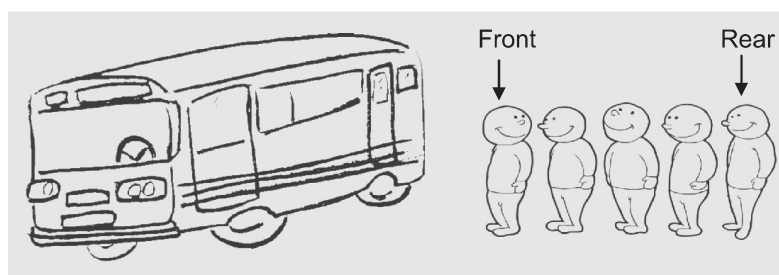


Fig. 5.1 Example of queue—passengers waiting at bus stop

To realize a queue as an abstract data type (ADT), we need a suitable data structure for storing the elements in the queue and the functions operating on it. The basic operations performed on the queue include adding and deleting an element, traversing the queue, checking whether the queue is full or empty, and finding who is at the front and who is at the rear ends.

A minimal set of operations on a queue is as follows:

1. `create()`—creates an empty queue, Q
2. `add(i, Q)`—adds the element i to the rear end of the queue, Q and returns the new queue
3. `delete(Q)`—takes out an element from the front end of the queue and returns the resulting queue
4. `getFront(Q)`—returns the element that is at the front position of the queue
5. `Is_Empty(Q)`—returns true if the queue is empty; otherwise returns false

The complete specification for the queue ADT is given in Algorithm 5.1.

ALGORITHM 5.1

```
class queue(element)
  declare create() → queue
  add(element, queue) → queue
  delete(queue) → queue
  getFront(queue) → queue
```

```

Is_Empty(queue) → Boolean;
For all Q ∈ queue, i ∈ element let
Is_Empty(create()) = true
Is_Empty(add(i,Q)) = false
delete(create()) = error
delete(add(i,Q)) =
    if Is_Empty(Q) then create
    else add(i, delete(Q))
getFront(create) = error
getFront(add(i, Q)) =
    if Is_Empty(Q) then i
    else getFront(Q)
end
end queue

```

Since a queue is a linear data structure, it can be implemented using either arrays or linked lists. For the former, we use static memory allocation and for the latter, we use dynamic memory allocation. Let us see how a queue can be implemented using arrays.

5.3 REALIZATION OF QUEUES USING ARRAYS

We already know that an array is not a suitable data structure for frequent insertion and deletion of data elements. Another drawback of arrays is that they use static memory allocation, and so they can store only a fixed number of elements. In many practical applications, we come across a situation where the size of the data set keeps changing by such frequent insertions and deletions. Let us see the implementation of the various operations on the queue using arrays.

Create This operation should create an empty queue. Here `max` is the maximum initial size that is defined.

```

#define max 50
int Queue[max];
int Front = Rear = -1;

```

In addition to a one-dimensional array `Queue`, we need two more variables, `Front` and `Rear`. This declaration creates an empty queue of size `max`. The two variables `Front` and `Rear` are initialized to represent an empty queue. In general, it is suitable to set `Front` to one position behind the actual front of the queue and set the rear to the last element in the queue. Thus, the condition `Front = Rear` indicates an empty queue. As our array index ranges between 0 and $(\text{max} - 1)$, the front and rear are initialized to -1 .

Is_Empty This operation checks whether the queue is empty or not. This is confirmed by comparing the values of `Front` and `Rear`. If `Front = Rear`, then `Is_Empty` returns true, else returns false.

```

bool Is_Empty()
{
    if(Front == Rear)
        return 1;
}

```

```

    else
        return 0;
}

```

Is_Full In the definition of the queue ADT, the function for checking the `Queue_Full` condition is not included. When we go in for an array implementation, due to its fixed size, we need to check the state of the queue for being full. It is recommended that before we delete an element from the queue, we must check whether the queue is empty or not. Similarly, before insertion, the queue must be checked for the `Queue_Full` state. When `Rear` points to the last location of the array, it indicates that the queue is full, that is, there is no space to accommodate any more elements.

```

bool Is_Full()
{
    if(Rear == max - 1)
        return 1;
    else
        return 0;
}

```

Add This operation adds an element in the queue if it is not full. As `Rear` points to the last element of the queue, the new element is added at the $(\text{rear} + 1)^{\text{th}}$ location.

```

void Add(int Element)
{
    if(Is_Full())
        cout << "Error, Queue is full";
    else
        Queue[++Rear] = Element;
}

```

Delete This operation deletes an element from the front of the queue and sets `Front` to point to the next element. `Front` can be initialized to one position less than the actual front. We should first increment the value of `Front` and then remove the element.

```

int Delete()
{
    if(Is_Empty())
        cout << "Sorry, queue is Empty";
    else
        return(Queue[++Front]);
}

```

getFront The operation `getFront` returns the element at the front, but unlike `delete`, this does not update the value of `Front`.

```

int getFront()
{
    if(Is_Empty())
        cout << "Sorry, queue is Empty";
}

```

```

        else
            return(Queue[Front + 1]);
    }

```

Program Code 5.1 shows one way of realization of the queue ADT using arrays.

PROGRAM CODE 5.1

```

//Queue ADT
class queue
{
    private:
        int Rear, Front;
        int Queue[50];
        int max;
        int Size;
    public:
        queue()
        {
            Size = 0; max = 50;
            Rear = Front = -1 ;
        }
        int Is_Empty();
        int Is_Full();
        void Add(int Element);
        int Delete();
        int getFront();
};

int queue :: Is_Empty()
{
    if(Front == Rear)
        return 1;
    else
        return 0;
}

int queue :: Is_Full()
{
    if(Rear == max - 1)
        return 1;
    else
        return 0;
}

void queue :: Add(int Element)

```

```

{
    if(!Is_Full())
        Queue[++Rear] = Element;
    Size++;
}

int queue :: Delete()
{
    if(!Is_Empty())
    {
        Size--;
        return(Queue[++Front]);
    }
}

int queue :: getFront()
{
    if(!Is_Empty())
        return(Queue[Front + 1]);
}

```

This implementation of queues using arrays has some flaws in it. Let us discuss these flaws through Program Code 5.2.

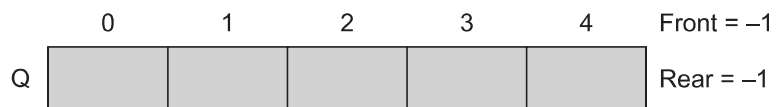
PROGRAM CODE 5.2

```

void main(void)
{
    queue Q;
    Q.Add(11);
    Q.Add(12);
    Q.Add(13);
    cout << Q.Delete() << endl;
    Q.Add(14);
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    Q.Add(15);
    Q.Add(16);
    cout << Q.Delete() << endl;
}

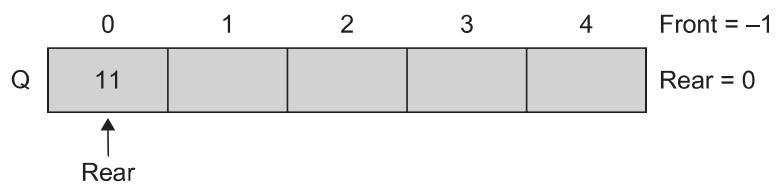
```

Let Q be an empty queue with $\text{Front} = \text{Rear} = -1$. Let $\text{max} = 5$.

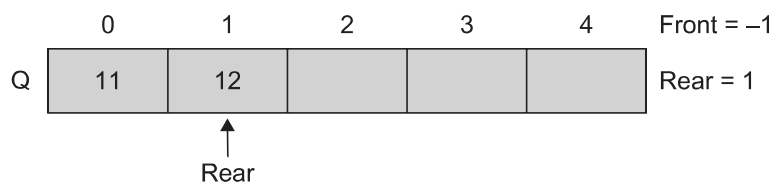


Consider the following statements:

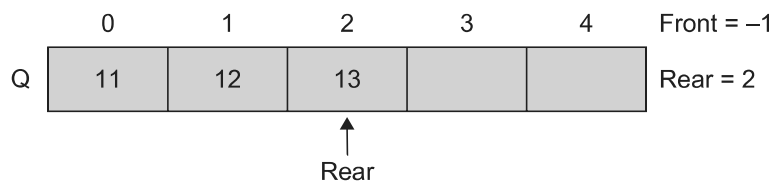
1. $Q.\text{Add}(11)$



2. $Q.\text{Add}(12)$

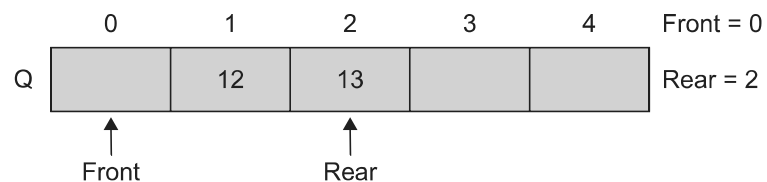


3. $Q.\text{Add}(13)$

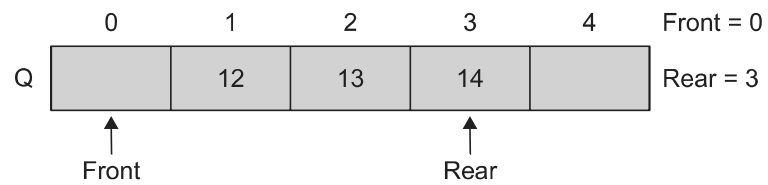


4. $A = Q.\text{Delete}()$

Here, $A = Q[++\text{Front}] = Q[0] = 11$

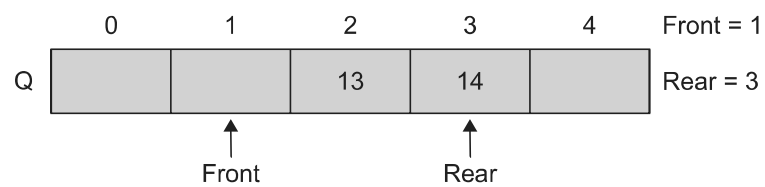


5. `Q.Add(14)`



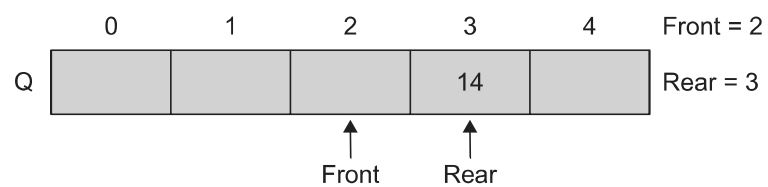
6. `A = Q.Delete()`

`A = Q[++ Front] = Q [1] = 12`

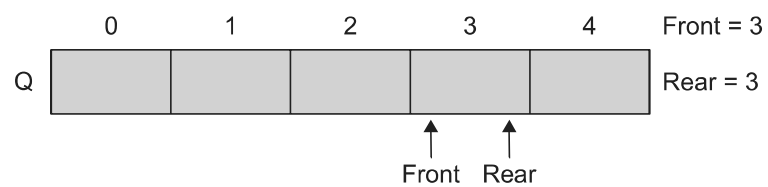


7. `A = Q.Delete()`

`A = 13`



8. `A = Q.Delete()`

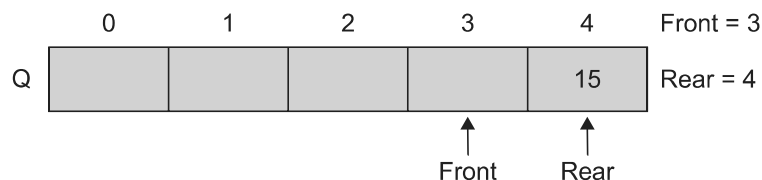


9. `A = Q.Delete()`

Here we get the `Queue_empty` error condition as `Front = Rear = 3`

Let us execute a few more statements.

10. `Q.Add(15)`



11. `Q.Add(16)`

This statement will generate the message `Queue_Full` because `Rear = 4`. If one carefully observes whether the queue is really full, it actually is not. The `Queue_Full` state should have five elements in it, whereas currently, there is only one element in the queue. This means that the implementation needs to be modified.

The precision of this implementation may be established in a manner similar to that used for stacks. With this setup, notice that unless the front regularly catches up with the rear and both the pointers are reset to zero, the `Queue_Full` condition does not necessarily indicate that it is full. One obvious thing to do when `Queue_Full` is signalled is to move the entire queue to the left so that the first element is again at the 0th location and `Front = -1`. This is obviously not a feasible solution as it is time consuming and involves a lot of data movement. This becomes impractical, especially when the queue is of a large size. The queue we have discussed so far is called the *linear queue*. There are two solutions to this problem: one is using a circular queue and the other is using a linked organization for realization of the queue. Let us discuss circular queues in Section 5.4.

5.4 CIRCULAR QUEUE

From the demonstration of the execution of a few `push` and `pop` operations it can be concluded that the linear queues using arrays have certain drawbacks listed as follows:

1. The linear queue is of a fixed size. So the user does not have the flexibility to dynamically change the size of the queue.
2. An arbitrarily declared maximum size of queues leads to poor utilization of memory. For example, the queue is declared of size 1000 and only 20 of them are used.
3. We need to write a suitable code to make the *front* regularly catch up with the *rear* and reset both. Array implementation of linear queues leads to the `Queue_Full` state even though the queue is not actually full.
4. To avoid this, when `Queue_Full` is signalled, we need to rewind the entire queue to the original start location (if there are empty locations) so that the first element is at the 0th location and `Front` is set to `-1`. Such movement of data is an efficient way to avoid this drawback.

The technique that essentially allows the queue to wraparound upon reaching the end of the array eliminates these drawbacks. Such a technique which allows the queues to

wraparound from end to start is called a *circular queue*. Virtually, we want the insertion process and the rear to wraparound the queue.

Hence, a more efficient queue representation is obtained by implementing the array Q as circular. Here, as we go on adding elements to the queue and reach the end of the array, the next element is stored in the first slot of the array if it is empty. Suppose the queue Q is of size n . Now, if we go on adding elements in the queue, we may reach the location $n - 1$. If it is not circular, no more elements can be added even though there are empty locations at the front of the array. Instead, if there are empty locations at the front, using a circular queue we can add elements at that location rather than signalling an error as the queue is full or is shifting the data.

The empty slots will be filled with new incoming elements even though $\text{Rear} = n - 1$. Hence, the circular queue allows us to continue adding elements even though we have reached the end of the array. The queue is said to be full only when there are n elements in the queue. The pictorial representation of a circular queue is shown in Figs 5.2(a) and 5.2(b).

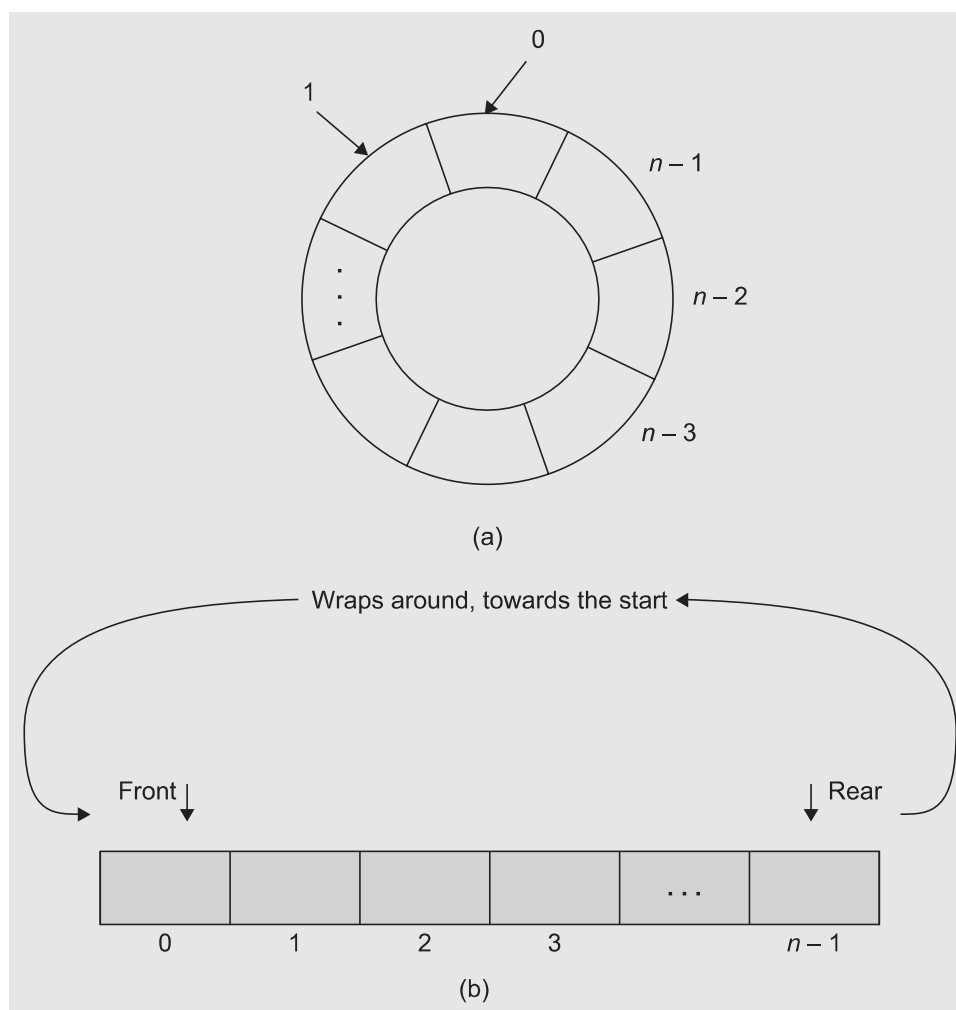


Fig. 5.2 Circular queue (a) Conceptual view (b) Physical view

Let us consider the queue Q which is of size n . We have already studied the operations on linear queues using arrays. We also studied its corresponding functions in the C++ language. Let us see whether the same functions can be used for circular queues. In a circular queue, when the *rear* is $n - 1$ and a new element is to be added, the *rear* should be set to 0.

Initially, both the *front* and the *rear* are set to -1 . The value of *front* will always be one less than that of the actual *front*. The functions to add and delete elements are rewritten with a few modifications in Program Code 5.3.

PROGRAM CODE 5.3

```
#include<iostream.h>
class Cqueue
{
    private:
        int Rear, Front;
        int Queue[50];
        int Max;
        int Size;
    public:
        Cqueue() {Size = 0; Max = 50; Rear = Front = -1;}
        int Empty();
        int Full();
        void Add(int Element);
        int Delete();
        int getFront();
};

int Cqueue :: Empty()
{
    if(Front == Rear)
        return 1;
    else
        return 0;
}

int Cqueue :: Full()
{
    if(Rear == Front)
        return 1;
    else
        return 0;
}
```

```
void Cqueue :: Add(int Element)
{
    if(!Full())
        Rear = (Rear + 1) % Max;
    Queue[Rear] = Element;
    Size++;
}

int Cqueue :: Delete()
{
    if(!Empty())
        Front = (Front + 1) % Max;
    Size--;
    return(Queue[Front]);
}

int Cqueue :: getFront()
{
    int Temp;
    if(!Empty())
        Temp = (Front + 1) % Max;
    return(Queue[Temp]);
}

void main(void)
{
    Cqueue Q;
    Q.Add(11);
    Q.Add(12);
    Q.Add(13);
    cout << Q.Delete() << endl;
    Q.Add(14);
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    Q.Add(15);
    Q.Add(16);
    cout << Q.Delete() << endl;
}
```

The implementation of a circular queue using an array is provided in Program Code 5.3. Let us see its working with an example. Consider $\text{max} = 5$ and initially, $\text{Front} = \text{Rear} = 0$. The iterations are shown in Table 5.1.

Table 5.1 Implementation of circular queue

0	1	2	3	4	Front	Rear	Action
					0	0	Q_Empty
	11				0	1	Insert 11
	11	12			0	2	Insert 12
	11	12	13		0	3	Insert 13
	11	12	13	14	0	4	Insert 14
	11	12	13	14	0		Insert 15 Can't insert 15 as Q_Full since $\text{Rear} = (4 + 1)\%5 = 0$ which is equal to Front
	–	12	13	14	1	4	Delete
		–	13	14	2	4	Delete
			–	14	3	4	Delete
				–	4	4	Delete Can't delete as $\text{Front} = \text{Rear}$ makes Q_Empty

To check the `Queue_Full` and `Queue_Empty` conditions, we need to check whether the values of `Front` and `Rear` are equal. In the programming languages C/C++, the array index varies from 0 to $n - 1$, so that one location of the circular queue always remains unused. Such is not the case in languages such as Pascal. Hence, in a circular queue that uses arrays in C/C++, we can store $n - 1$ elements, where n is declared as the size of the array. Hence, for storing n elements, we should declare the array of size $n + 1$.

5.4.1 Advantages of Using Circular Queues

The following are the merits of using circular queues:

1. By using circular queues, data shifting is avoided as the *front* and *rear* are modified by using the `mod()` function. The `mod()` operation wraps the queue back to its beginning.
2. If the number of elements to be stored in the queue is fixed (i.e., if the queue size is specific), the circular queue is advantageous.
3. Many practical applications such as printer queue, priority queue, and simulations use the circular queue.

5.5 MULTI-QUEUES

If more number of queues is required to be implemented, then an efficient data structure to handle multiple queues is required. It is possible to utilize all the available spaces in a single array. When more than two queues, say n , are represented sequentially, we can

divide the available memory $A[\text{size}]$ into n segments and allocate these segments to n queues, one to each. For each queue i we shall use $\text{Front}[i]$ and $\text{Rear}[i]$. We shall use the condition $\text{Front}[i] = \text{Rear}[i]$ if and only if the i^{th} queue is empty, and the condition $\text{Rear}[i] = \text{Front}[i] + 1$ if and only if the i^{th} queue is full.

If we want five queues, then we can divide the array $A[100]$ into equal parts of 20 and initialize *front* and *rear* for each queue, that is, $\text{Front}[0] = \text{Rear}[0] = 0$ and $\text{Front}[1] = \text{Rear}[1] = 20$, and so on for other queues (Fig. 5.3).

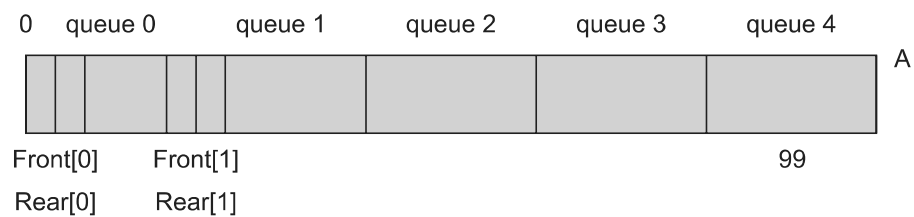


Fig. 5.3 A multi-queue

After adding elements 5 and 8 in the second queue, the resultant queue will be as in Fig. 5.4.

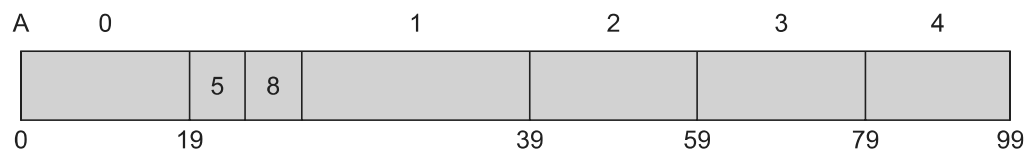


Fig. 5.4 Queue in Fig. 5.3 after addition of elements

5.6 DEQUE

The word *deque* is a short form of double-ended queue. It is pronounced as 'deck'. *Deque* defines a data structure where elements can be added or deleted at either the front end or the rear end, but no changes can be made elsewhere in the list. Thus, *deque* is a generalization of both a stack and a queue. It supports both stack-like and queue-like capabilities. It is a sequential container that is optimized for fast index-based access and efficient insertion at either of its ends. Deque can be implemented as either a continuous deque or as a linked deque. Figure 5.5 shows the representation of a deque.

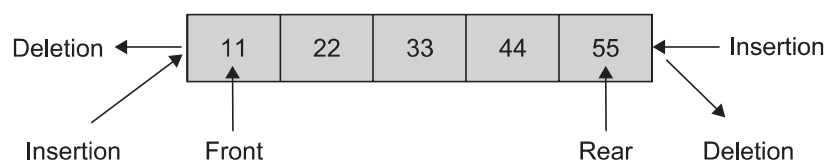


Fig. 5.5 Representation of a deque

The *deque ADT* combines the characteristics of stacks and queues. Similar to stacks and queues, a deque permits the elements to be accessed only at the ends. However, a deque allows elements to be added at and removed from either end. We can refer to the operations supported by the deque as `EnqueueFront`, `EnqueueRear`, `DequeueFront`, and `DequeueRear`. When we complete a formal description of the deque and then implement it using a dynamic, linked implementation, we can use it to implement both stacks and queues, thus achieving significant code reuse.

The following are the four operations associated with deque:

1. `EnqueueFront()`—adds elements at the front end of the queue
2. `EnqueueRear()`—adds elements at the rear end of the queue
3. `DequeueFront()`—deletes elements from the front end of the queue
4. `DequeueRear()`—deletes elements from the rear end of the queue

For stack implementation using deque, `EnqueueFront` and `DequeueFront` are used as `push` and `pop` functions, respectively.

Applications of deque Deque is useful where the data to be stored has to be ordered, compact storage is needed, and the retrieval of data elements has to be faster.

Variations of deque We can have two variations of a deque: the *input-restricted deque* and the *output-restricted deque*. The output-restricted deque allows deletions from only one end and the input-restricted deque allows insertions only at one end.

The functions to operate an output-restricted deque could be as follows:

`DequeueFront()` (or `DequeueRight()`), `EnqueueFront()`, and `EnqueueRear()`

The functions to operate an input-restricted deque are as follows:

`DequeueFront()`, `DequeueRight()`, and `EnqueueFront()` (or `EnqueueRear()`)

5.7 PRIORITY QUEUE

A *priority queue* is a collection of a finite number of prioritized elements. *Priority queues* are those in which we can insert or delete elements from any position based on some fundamental ordering of the elements. Elements can be inserted in any order in a priority queue, but when an element is removed from the priority queue, it is always the one with the highest priority.

In other words a priority queue is a collection of elements where the elements are stored according to their priority levels. The order in which the elements should be removed is decided by the priority of the element. The following rules are applied to maintain a priority queue:

1. The element with a higher priority is processed before any element of lower priority.
2. If there were elements with the same priority, then the element added first in the queue would get processed first.